LECTURE 4

MLflow for GenAI

Tracing, evaluation and observability

4 November 2026

LLMOps with Databricks



LECTURE 4

Agenda

01 **MLflow components**
Tracking, registry and tracing

02 **Storage and governance**
Regular ML, GenAI and Unity Catalog

03 **Traces and spans**
Inspect and instrument agent execution

04 **Evaluation**
Test behavior, define scorers and inspect failures

05 **Tracking and registration**
Prepare an agent for Model Serving



MLflow components



MLflow components

01

Experiment tracking

Main use: machine learning. Compare training runs, parameters, metrics and artifacts.

Also used to log agent artifacts for Model Serving.

02

Model registry

Manage model versions and aliases in Unity Catalog. Share models across workspaces.

ML models and agents served as models.

03

MLflow tracing

Understand what happens inside an agent. OpenTelemetry-compatible and closely tied to evaluation.

Introduced in MLflow 2.14.0 · June 2024

For an agent behind a Model Serving endpoint, also log the agent and register a model version.



LLM observability tools also support evaluation

Tool / ecosystem	Observability	Evaluation
Pydantic Logfire + pydantic-evals	✓	✓
MLflow	✓	✓
Arize Phoenix	✓	✓
Opik (Comet)	✓	✓
Langfuse	✓	✓

Human review and feedback are also supported by MLflow, Phoenix, Opik and Langfuse.

> **Observability and evaluation belong in the same workflow.**



Model flavors and payloads

Classic ML

sklearn · statsmodels
pytorch · catboost
spacy · lightgbm
xgboost · tensorflow

Custom pyfunc.PythonModel

REQUEST ALTERNATIVES

```
{"dataframe_split": {
  "columns": ["x"], "data": [[1]]}}
{"dataframe_records": [{"x": 1}]}

{"instances": [1.0, 2.0, 5.0]}
{"inputs": [["cheese"], ["Crackers"]]}
```

instances / inputs: tensor models

```
{"predictions": [[120.5]]}
```

AI agents

dspy
langchain
llama_index

Custom pyfunc.ResponsesAgent

RESPONSES-STYLE INPUT

```
{"input": [{
  "role": "user",
  "content": "Find new papers"
}]}
```

ResponsesAgent defines the request and response schema.

Other agent flavors: check the model signature; chat format is not universal.

Foundation models

transformers

Deploy supported model families and architectures

Interface **Depends on the task**

SUPPORTED ENDPOINT TASKS

Chat · Embeddings · Completions

```
{"messages": [{
  "role": "user", "content": "Hi"
}]}

{"input": "Text to embed"}

{"prompt": "Once upon a time"}
```



Storage and governance

Regular ML and GenAI experiments

REGULAR ML

Training experiments

Workspace

Experiments with training runs,
parameters and metrics

Unity Catalog

Registered ML models:
versions and aliases

GENAI

Tracing and evaluation

Experiment

Organizes agent traces
and evaluation results

Unity Catalog

Trace data can be stored in UC
with governed access

Model Serving is the overlap: log the agent in an MLflow run, then register it in UC.



Why GenAI changes the data sensitivity

TRADITIONAL ML

Often aggregate information

learning_rate	0.01
max_depth	8
validation_f1	0.89

Illustrative run metadata

GENAI

The content of a request

“Summarise this confidential customer case…”

Retrieved document excerpts

Tool arguments and the full response

A trace can become another copy of protected business data.

> Traditional ML artifacts and logged samples can also contain sensitive data.



Workspace administration and trace access

Workspace-managed traces

Workspace admin → Experiment ACLs → **Trace contents**

Workspace admins can manage experiment permissions and potentially read sensitive payloads.

UC-backed traces

Application team → UC privileges → **Trace tables**

Govern trace access through catalog, schema and table permissions.

Ownership matters. Workspace admins may also own a catalog or hold UC privileges.



Registered models in Unity Catalog

Workspace A

Develop and compare

Log and evaluate an agent version

↔

UNITY CATALOG

catalog.schema.arxiv_agent

Version 1 | Version 2 | Version 3

Governed model name and aliases

↔

Workspace B

Load and deploy

Use an authorised model version

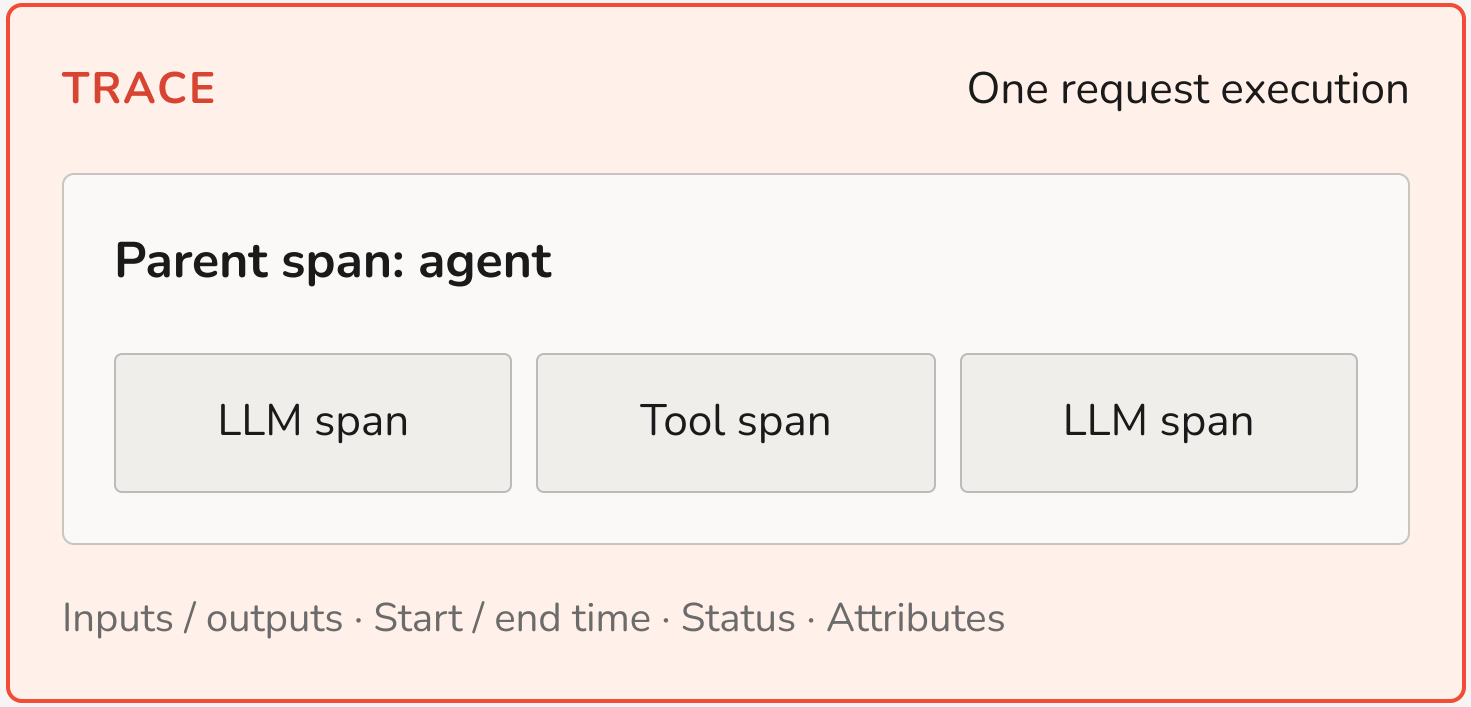
Same metastore, appropriate privileges, and permitted workspace–catalog bindings.



Traces and spans



A trace records one execution



A span is one operation

Nesting preserves which operation called which.

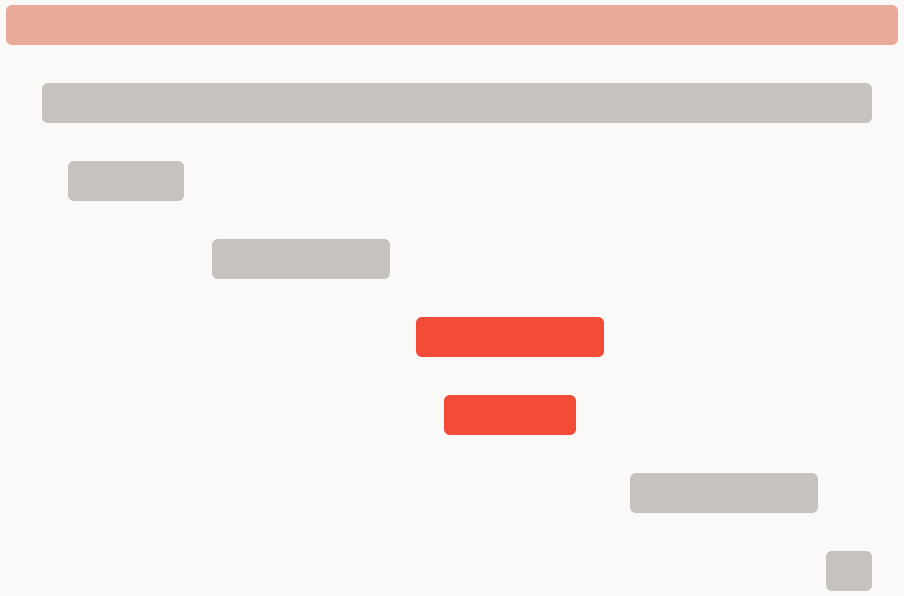
A session links requests

One conversation can contain several traces.

One request through the arXiv agent

One request, with nested operations you can inspect.

predict_stream	AGENT
call_and_run_tools	CHAIN
memory_load	RETRIEVER
LLM call	LLM
execute_tool	TOOL
search_papers	retrieval work
LLM call	LLM
memory_save	CHAIN



Illustrative hierarchy and timing. The actual spans depend on the executed path and integrations.

What a trace helps us investigate

01

Slow response

Which span dominates the request? Is the agent making repeated tool calls?

02

Poor answer

What did retrieval return? What did the model receive? Did memory load the expected history?

03

Unexpected cost

How many model calls ran? Where available, compare token usage by span and version.

Execution succeeded. That alone does not tell us whether the answer was useful.



Instrumentation choices

Automatic instrumentation

Use the integration for your model or framework to capture supported operations.

Supported framework operations

@mlflow.trace

Instrument your own functions and choose meaningful span names and types.

Your own functions

mlflow.start_span

Control the scope of a span and explicitly record inputs or outputs.

A precise section of code



UC trace setup

```
from mlflow.entities.trace_location import UnityCatalog

experiment = mlflow.set_experiment(
    experiment_name=cfg.experiment_name,
    trace_location=UnityCatalog(
        catalog_name=cfg.catalog,
        schema_name=cfg.schema,
        table_prefix="llmops",
    ),
)
```

NOTEBOOK 4.0

The destination is configuration

The experiment is the entry point.

Catalog, schema and prefix identify the trace tables.

The full notebook checks the binding and grants access.



Permissions and deployment checks

Identity	Access to check
Human reader	Intended trace data and SQL warehouse access
Application principal	Parent USE grants and required trace-table privileges
Unrelated user	No unintended access through inherited grants

WRITE

Generate a trace as the application

READ

Inspect it as the intended reader

> Notebook 4.0 grants table SELECT / MODIFY and view SELECT. Parent USE privileges must also exist.



A parent span and two child spans

```
@mlflow.trace(span_type=SpanType.CHAIN)
def process_query(user_query: str) -> str:
    results = search_database(user_query)
    prompt = f"Question: {user_query}\nResults: {results}"
    return call_llm(prompt)

# search_database uses @mlflow.trace(span_type=SpanType.TOOL)
# call_llm uses @mlflow.trace(span_type=SpanType.LLM)
```

NOTEBOOK 4.1

One parent, two children

process_query

- search_database
- call_llm

Decorators preserve the nesting between calls.



Metadata connects requests to versions

```
mlflow.update_current_trace(
    metadata={
        "mlflow.trace.session": session_id,
        "environment": env,
    },
    tags={"git_sha": git_sha, "model_version": version},
    client_request_id=request_id,
)
```

REQUEST CONTEXT

Three useful links

Request ID
One client request

Session ID
Related conversation turns

Version
The implementation that ran



Demo: inspect the agent execution

01

Configure & instrument

4.0–4.1: verify the UC destination, generate a trace and inspect nesting.

02

Run the agent

4.2: ask for papers, then follow up with the same session ID.

03

Explain the execution

Find the memory load, tool call and final response.

> Which evidence in the trace supports the answer?



Evaluation



Evaluation data starts with expected behavior

ONE ARXIV EVALUATION CASE

“Any newer papers on this topic?”

Context

The previous topic and the papers already shown

Expected behavior

Use the conversation, retrieve relevant newer papers, and acknowledge missing evidence.

Build a reviewed dataset

- Normal requests and meaningful edge cases
- Prior turns or evidence where needed
- Explicit expectations
- Regression cases from real failures

EVALUATION

Metrics define what; scorers define how

Metrics define what to measure. Scorers define how.

Code-based scorers

Best for

Format, length, tool calls

Strength

Fast, low-cost and repeatable

Limitation

Limited nuance and context

Model-based scorers

Best for

Relevance and contextual quality

Strength

Flexible for open-ended outputs

Limitation

Variable scores: calibrate with humans

Human scorers

Best for

Business expectations and edge cases

Strength

Expert judgment and calibration

Limitation

Slow and costly to scale



EVALUATION

MLflow scorers: choosing the evidence

Scorer	What does it evaluate?	Evidence
RelevanceToQuery	Does it address the request?	Input and response
Correctness	Does it support expected facts?	Reference expectations
RetrievalGroundedness	Does retrieved evidence support it?	Retrieval spans
Guidelines	Does it follow our criteria?	Inputs, outputs and rules

> Choose a scorer that can see the evidence needed for the judgment.



EVALUATION

Guidelines and custom judges

GUIDELINES

Pass / fail

“When earlier attempts failed, acknowledge them and offer a new approach.”

Shared natural-language rules across examples.

CUSTOM JUDGE

A task-specific rubric

1. Repeats a failed suggestion
2. Acknowledges, but offers no useful next step
3. Acknowledges and gives a relevant next step

Custom instructions, template inputs and output type.

> Both approaches need calibration against human feedback.

EVALUATION

A guideline for conversation context

```
from mlflow.genai.scorers import Guidelines

context_guideline = Guidelines(
    name="acknowledges_previous_attempts",
    guidelines=[
        "If the user says earlier attempts failed, acknowledge "
        "those attempts and offer a new approach or escalation."
    ],
    model=JUDGE_MODEL,
)
```

GUIDELINES

A conditional rule

Only apply the expectation when the user describes failed attempts.

The judge needs the relevant conversation context.



EVALUATION

A custom judge with an explicit rubric

```
from mlflow.genai.judges import make_judge

context_judge = make_judge(
    name="context_handling",
    instructions=(
        "Given {{ inputs }} and {{ outputs }}, score 1–3: "
        "1 = repeats a failed suggestion; "
        "2 = acknowledges failure but gives no useful next step; "
        "3 = acknowledges failure and offers a relevant next step."
    ),
    feedback_value_type=int, model=JUDGE_MODEL,
)
```

CUSTOM JUDGE

A defined scale

1 Repeats a failed suggestion

2 Acknowledges, but does not help

3 Offers a useful next step

EVALUATION

Our course scorers: useful but incomplete

Our current checks

- ✓ Under 350 words
- ✓ Polite tone
- ✓ Engaging opening

? Does it represent the paper correctly?

A short, polite post can still misstate the research.

> Add factual support and task-specific expectations. Keyword matching alone cannot establish correctness.



Demo: evaluate and inspect a failure

```
results = mlflow.genai.evaluate(
    predict_fn=predict_fn,
    data=eval_data,
    scorers=[
        word_count_check,
        polite_tone_guideline,
        hook_in_post_guideline,
    ],
)
```

NOTEBOOKS 4.3–4.4

Read individual cases

Response

Scorer feedback

Underlying trace

What did the scorer miss?



EVALUATION

Judge reliability needs human calibration

A shared rubric

Give the judge and reviewers the same criteria and context.

Human feedback

Judge feedback

Inspect disagreements

False passes

False failures

Ambiguous rules

Missing evidence

Check fresh examples

Reassess on cases that did not shape the judge.

LLM judges provide scalable feedback. Their scores still need scrutiny.

Tracking and registration

Tracking and registration complete the workflow

01

Evaluate

Inspect individual cases and decide whether the version is ready.

→

02

Track & log

Record metrics and log the configured agent artifact in an MLflow run.

→

03

Register

Create a UC model version and update its alias deliberately.

> Recording evaluation metrics does not automatically enforce a release gate.



The agent artifact for next week

```
with mlflow.start_run(run_name="arxiv-agent"):
    model_info = mlflow.pyfunc.log_model(
        name="agent", python_model=agent,
        input_example=test_request, model_config=model_config,
    )
    mlflow.log_metrics(results.metrics)

registered_model = mlflow.register_model(
    model_uri=model_info.model_uri,
    name=f"{cfg.catalog}.{cfg.schema}.arxiv_agent",
)
```

NOTEBOOK 4.4

Ready for the next step

The run records metrics and the agent artifact.

The registry gives that artifact a model version.

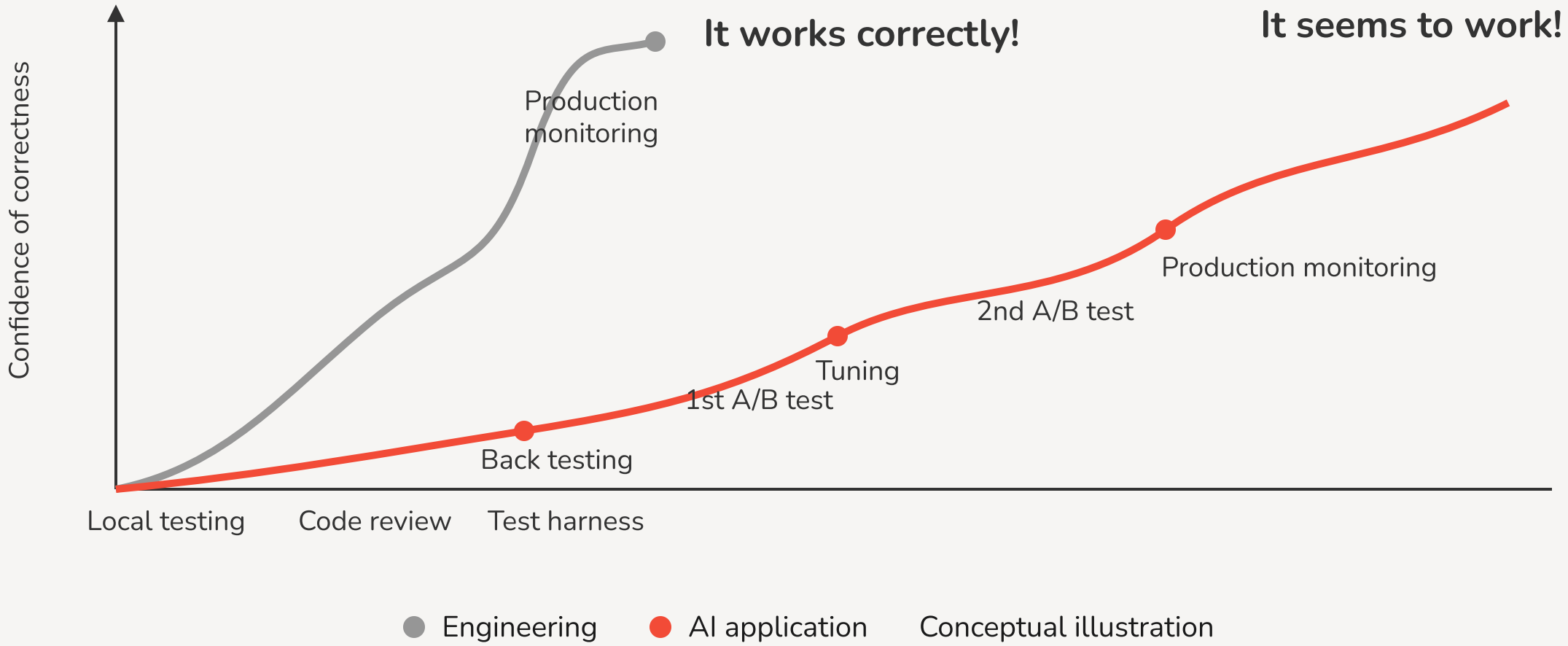
Registration does not deploy an endpoint.



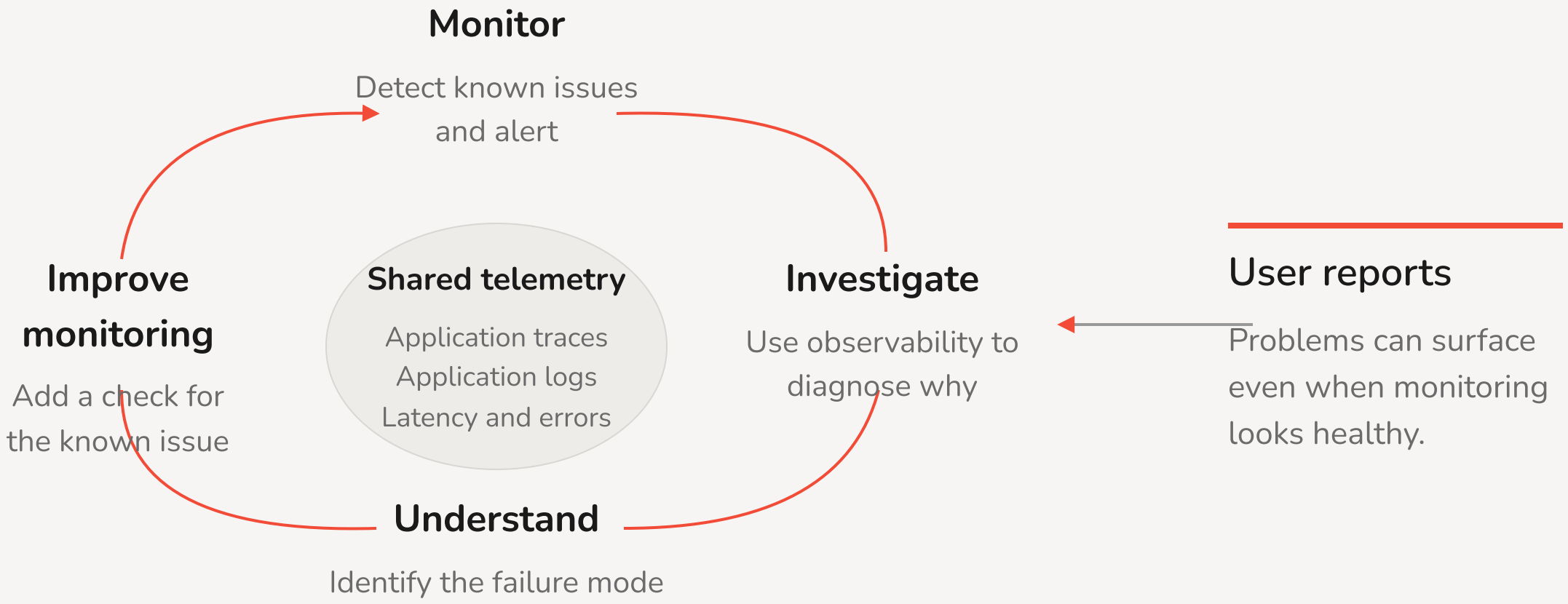
Production observability



System lifecycle starts after deployment



Monitoring vs observability



Easy checks can miss a real problem

A passing score does not guarantee a useful response.

The exchange

User

“Same issue as last month”

System

“Can you describe the problem?”

The evaluation

Check	Method	Result
Politeness	LLM-based	✓ Pass
Relevance	LLM-based	✓ Pass
≤ 300 characters	Code-based	✓ Pass

THE MISSING CHECK

Did the response use the **previous conversation?**



Evaluate the response in context

01

Returning issue

User
“Same issue as last month”

System
“Can you describe the problem?”

Expected
Reference previous tickets and confirm whether the issue is related.

× Unsatisfactory

Context retrieval

No

02

Quick question

User
“Just a quick question about settings”

System
“Sure, which settings are you looking at? I can walk you through it.”

Expected
Respond proportionally and ask a clarifying question.

✓ Satisfactory

Context retrieval

Not applicable

03

Repeated suggestion

User
“Tried everything you suggested”

System
“Have you tried restarting?”

Expected
Acknowledge previous attempts, escalate or try a new approach.

× Unsatisfactory

Context retrieval

No

> Trace information reveals whether the system retrieved and used the right context.

Observability and evaluation work together

Observability

What happened, and why?

Inputs, outputs, retrieval and tool calls

Latency, token usage and cost

Evaluation

How well did it perform?

Correctness, relevance and safety

Code checks, human review and LLM judges

Traces provide context for scoring



Low scores guide investigation

Traces explain the execution. Evaluations assess its quality.

PRODUCTION OBSERVABILITY

Tiered evaluation in production

Cheap checks run broadly. Expensive evaluations run on a sample.

Tier 1: code-based checks

Coverage: every request

- Latency, token counts and cost
- Tool usage and error rates
- Format validation and rule violations
- Length checks and regex matching

Example: process new traces every 15 minutes.

Tier 2: LLM-based scorers

Coverage: sampled requests

- Relevance and correctness
- Tone and professionalism
- Random or stratified sampling
- Additional coverage of edge cases

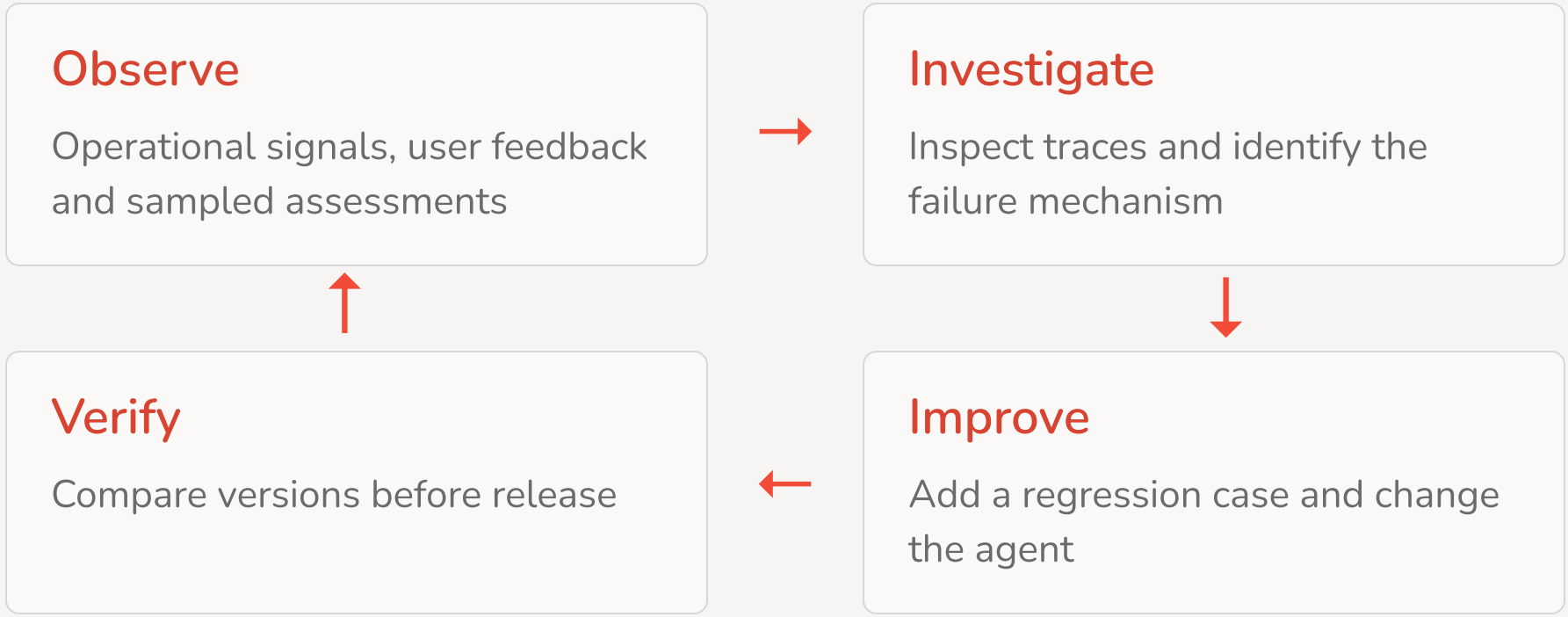
Example: evaluate a 5–10% sample hourly.

Illustrative rates and schedules. Run code checks in application code or jobs; MLflow automatic evaluation supports LLM judges.

> **Evaluate broadly with code. Use LLM scorers where context matters.**



Production failures become regression cases



Protect trace access and retention throughout the feedback loop.

Homework



HOMEWORK

Homework: trace, evaluate and explain

01 Instrument your agent

Configure its trace destination. Show a request with meaningful spans and version/session metadata.

02 Evaluate realistic behavior

Create a small reviewed dataset. Add a code check and a domain-specific judge with clear expectations.

03 Investigate a failure

Attach a sanitized trace example, explain the failure, and add a regression case.

04 Submit your work

Open a pull request with the implementation, evaluation findings and intended trace permissions.



Questions?

